

Technische Dokumentation

WPFTowerDefense2

WPF Tower Defense Spiel mit MVVM, SQLite und Entity Framework Core



Inhaltsverzeichnis

Inhaltsverzeichnis.....	2
1. Zweck und Umfang	3
2. Systemübersicht.....	3
3. UML-Klassendiagramm	4
4. Klassenbeschreibung.....	5
4.1 ViewModels	5
4.2 Datenklassen.....	6
5. Dateien und Projektstruktur	6
6. Datenbankdokumentation.....	7
6.1 Entity Framework Core	8
7. Wesentliche Programmfunktionen und Logik	8
7.1 Spielinitialisierung.....	8
7.2 Gegnerwellen.....	8
7.3 Turmplatzierung, Upgrade und Verkauf	8
7.4 Kampf- und Effektlogik.....	9
7.5 Gegnerbewegung und Lebenspunkte	9
7.6 Highscore und Game Over	9
8. Ressourcen und Assets.....	9

1. Zweck und Umfang

Dieses Dokument beschreibt die technische Struktur des Projekts WPFTowerDefense. Es dokumentiert die wichtigsten Klassen, Dateien, Datenbanktabellen und Programmabläufe. Die Beschreibung richtet sich an Entwicklerinnen und Entwickler, die das Projekt warten, erweitern oder testen.

- Klassen werden mit Aufgabe, Eigenschaften, Methoden und Abhängigkeiten beschrieben.
- Die Datenbankstruktur wird anhand der SQLite-Tabellen und Entity-Framework-Modelle dokumentiert.
- Zentrale Spielfunktionen wie Wellen, Gegnerbewegung, Turmangriffe, Effekte und Highscores werden im Ablauf erklärt.
- UML- und Abhängigkeitsdiagramme ergänzen die natürlichsprachliche Beschreibung.

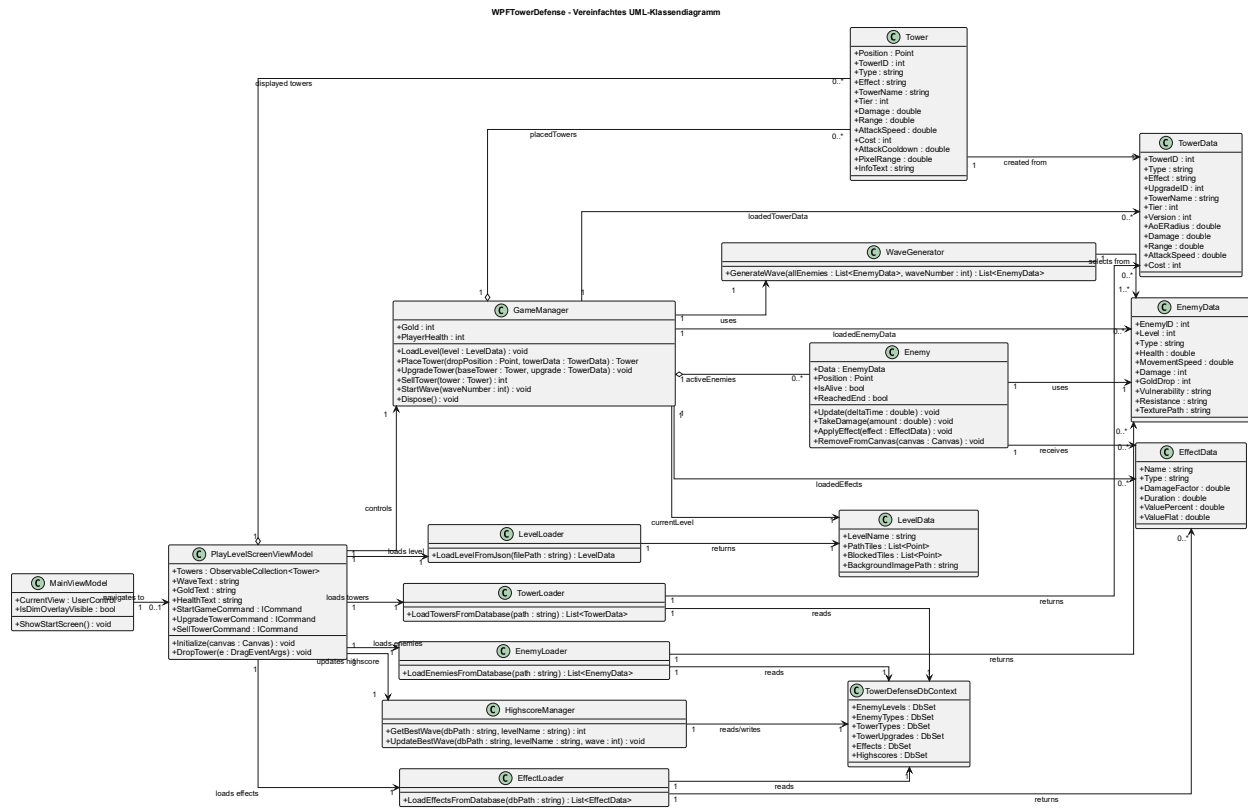
2. Systemübersicht

WPFTowerDefense ist eine WPF-Desktopanwendung. Die Oberfläche wird über XAML-Views aufgebaut, während ViewModels die UI-Zustände, Commands und Navigation steuern. Die Spiellogik liegt vor allem im GameManager, die Gegnerwellen werden vom WaveGenerator erzeugt. Statische Spieldaten wie Gegner, Türme, Effekte und Highscores werden aus der SQLite-Datenbank TD.db geladen.

Bereich	Technische Umsetzung	Wichtige Dateien
Präsentation	WPF Views mit DataContext und Bindings	Views/*.xaml, MainWindow.xaml
ViewModels	MVVM-Zustand, Commands, Navigation über Prism Events	ViewModels/*.cs
Spiellogik	Canvas-Rendering, Wellen, Gegner, Türme, Effekte	GameManager.cs, Enemy.cs, Tower.cs, WaveGenerator.cs
Datenzugriff	Entity Framework Core mit SQLite	Database/TowerDefenseDbContext.cs, Database/*Loader.cs
Datenmodelle	DTO-ähnliche Klassen für Spiel- und Leveldaten	GameLogic/*.cs
Assets	PNG-Ressourcen für Türme, Gegner und Map Tiles	Resources/*, Images/*

3. UML-Klassendiagramm

Das folgende Diagramm zeigt die wichtigsten Klassen und Paketbereiche. Es fasst die MVVM-Struktur, die Spiellogik und den Datenzugriff zusammen.



4. Klassenbeschreibung

Klasse	Zweck	Wesentliche Verantwortung	Abhängigkeiten
App	Startklasse der WPF-Anwendung.	Initialisiert die Anwendung und den Prism EventAggregator.	MainWindow, MainViewModel
MainWindow	Hauptfenster der Anwendung.	Hostet die aktuelle View und das Overlay.	MainViewModel
MainViewModel	Zentrale Navigationssteuerung.	Reagiert auf NavigationEvent, GameOverEvent und DimOverlayEvent und setzt CurrentView.	Views, ViewModels, Prism
ViewModelBase	Basisklasse für ViewModels.	Stellt EventAggregator bereit und erbt PropertyChanged-Funktionalität.	NotifyPropertyChanged
PlayLevelScreenViewModel	Steuert den Spielscreen.	Lädt Level, Türme, Gegner und Effekte; startet Wellen; verarbeitet Turmplatzierung, Upgrades und Verkauf.	GameManager, Loader, Events
GameManager	Zentrale Spiellogik.	Initialisiert Grid, verwaltet Gold/HP, platziert Türme, spawnet Gegner, berechnet Angriffe und Wellenende.	Canvas, Enemy, Tower, WaveGenerator
Enemy	Laufender Gegner im Spiel.	Bewegt sich entlang des Pfads, speichert Leben, zeichnet HealthBar und verarbeitet Effekte.	EnemyData, EffectData, Canvas
Tower	Platzierter Turm.	Speichert Werte wie Schaden, Reichweite, Angriffsgeschwindigkeit, Effekt und Upgrade-Informationen.	TowerData, NotifyPropertyChanged
WaveGenerator	Erzeugt Gegnerlisten für Wellen.	Wählt Gegnerlevel und Typen zufällig aus; fügt Bossgegner ab Welle 10 alle 5 Wellen hinzu.	EnemyData
PlayLevelInteraction	Attached Properties für View-Interaktion.	Verbindet Canvas-Initialisierung, Drag-and-Drop und MouseDown-Handling MVVM-konform mit dem ViewModel.	PlayLevelScreenViewModel
ActionCommand	ICommand-Implementierung.	Kapselt Execute- und CanExecute-Delegates für Button- und UI-Commands.	WPF Commanding
NotifyPropertyChanged	PropertyChanged-Basisklasse.	Benachrichtigt die UI bei Property-Änderungen.	INotifyPropertyChanged

4.1 ViewModels

ViewModel	Beschreibung
StartScreenViewModel	Stellt Commands für Start, Levelauswahl, Optionen, Game Guide und Exit bereit.
SelectLevelScreenViewModel	Verwaltet die Levelauswahl und Preview-Bilder und startet ein gewähltes Level per NavigationEvent.
OptionsScreenViewModel	Verwaltet Optionen wie Lautstärkeanzeige und Ruecknavigation.
GameGuideScreenViewModel	Stellt die Navigation aus der Spielanleitung zurück zum Start bereit.
GameOverScreenViewModel	Zeigt erreichte Welle, Levelname und Highscore-Informationen an.
ExitConfirmationWindowViewModel	Steuert die Bestätigung beim Beenden der Anwendung.

4.2 Datenklassen

Klasse	Abgebildete Daten	Besonderheiten
TowerData	TowerTypes und TowerUpgrades	Enthält Turmtyp, Effekt, UpgradeID, Tier, Version, AoE, Schaden, Reichweite, Angriffsgeschwindigkeit und Kosten.
EnemyData	EnemyTypes und EnemyLevels	Enthält Gegnerwerte und berechnet TexturePath; Boss verwendet Boss.png, normale Gegner Enemy{Type}.png.
EffectData	Effect	Definiert Slow, Burn, Stun oder None mit Dauer, Prozentwerten und Schadensfaktoren.
LevelData	Level-JSON-Dateien	Enthält LevelName, PathTiles, BlockedTiles und optional BackgroundImagePath.
PointListJsonConverter	Koordinatenlisten aus JSON	Konvertiert Arrays wie [x, y] in WPF-Point-Objekte.

5. Dateien und Projektstruktur

Pfad/Bereich	Inhalt	Abhängigkeiten
WPFTowerDefense.csproj	Projektdatei mit .NET 8 WPF, Ressourcen und NuGet-Paketen.	Microsoft.EntityFrameworkCore.Sqlite, Prism.Events
Views/*.xaml	UI-Definitionen für Start, Levelauswahl, Spiel, Optionen, Guide und Game Over.	ViewModels, Bindings, Commands
ViewModels/*.cs	Zustand und Logik der Oberfläche.	Common, Events, Database, GameManager
GameManager.cs	Hauptlogik des laufenden Spiels.	Canvas, Enemy, Tower, WaveGenerator, Loaderdaten
Enemy.cs / Tower.cs	Laufzeitobjekte für Gegner und Türme.	GameLogic-Datenklassen, WPF Canvas/Image
Database/*.cs	EF-Core-Kontext, Entity-Klassen und Loader.	SQLite-Datenbank TD.db
Database/TD.db	SQLite-Datenbank mit Gegnern, Türme, Effekten und Highscores.	Entity Framework Core
Levels/*.json	Leveldefinitionen mit Pfadkoordinaten.	LevelLoader, PointListJsonConverter
Resources/Enemies, Resources/Towers, Resources/MapTiles	Spielgrafiken als WPF-Ressourcen.	EnemyData.TexturePath, GameManager.LoadTexture, Tower.TexturePath

6. Datenbankdokumentation

Die Anwendung nutzt SQLite. Der Entity-Framework-Kontext TowerDefenseDbContext verbindet sich über UseSqlite mit dem Pfad Database/TD.db.

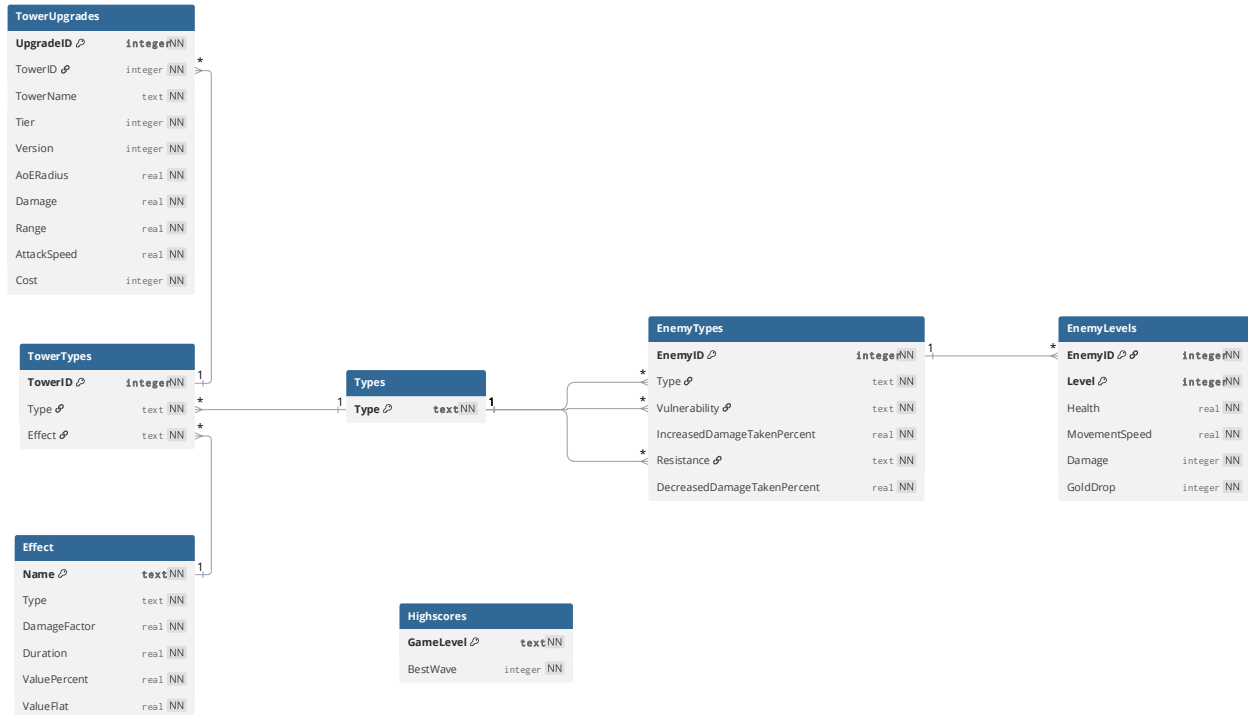


Tabelle	Primärschlüssel	Beschreibung	Abhängigkeiten
Types	Type	Stammtabelle für Typnamen wie Fire, Water, Grass, Normal oder Boss.	EnemyTypes, TowerTypes
EnemyTypes	EnemyID	Definiert Gegnertyp, Verwundbarkeit, Resistenz und Schadensmodifikatoren.	EnemyLevels, Types
EnemyLevels	EnemyID + Level	Definiert Levelwerte eines Gegners: Health, MovementSpeed, Damage und GoldDrop.	EnemyTypes
TowerTypes	TowerID	Definiert Turmtyp und zugeordneten Effekt.	Types, Effect, TowerUpgrades
TowerUpgrades	UpgradeID	Definiert konkrete Turmwerte pro Tier und Version.	TowerTypes
Effect	Name	Definiert Effekttypen, Dauer, Schadensfaktor und Prozentwerte.	TowerTypes, Enemy.ApplyEffect
Highscores	GameLevel	Speichert die beste erreichte Welle je Level.	HighscoreManager

6.1 Entity Framework Core

- TowerDefenseDbContext definiert DbSet-Properties für EnemyLevels, EnemyTypes, TowerTypes, TowerUpgrades, Effects und Highscores.
- OnConfiguring verwendet UseSqlite mit dem übergebenen Datenbankpfad.
- OnModelCreating setzt Tabellennamen und Primärschlüssel. EnemyLevels besitzt einen zusammengesetzten Schlüssel aus EnemyID und Level.
- Loader-Klassen projizieren Datenbankzeilen in GameLogic-Datenklassen, damit die Spiellogik nicht direkt mit EF-Entities arbeiten muss.

7. Wesentliche Programmfunktionen und Logik

7.1 Spielinitialisierung

Beim Öffnen des Spielscreens ruft PlayLevelInteraction die Initialize-Methode des PlayLevelScreenViewModel auf. Dort werden Gegner, Effekte und Türme aus der Datenbank geladen. Das Level wird aus einer JSON-Datei geladen. Anschliessend initialisiert der GameManager das Spielfeld mit 16 x 8 Tiles und wählt passende Texturen für Pfad- und Bodenfelder.

7.2 Gegnerwellen

- StartNextWave erhöht den aktuellen Wellenzähler und ruft GameManager.StartWave auf.
- WaveGenerator.GenerateWave berechnet die Gegneranzahl mit $5 + \text{waveNumber} * 2$.
- Ab Welle 6 werden die Typen Grass, Fire, Water und Normal gemischt; bis Welle 5 erscheinen hauptsächlich Grass-Gegner.
- Ab Welle 10 erscheint alle 5 Wellen ein Boss. Der Boss-Level berechnet sich aus der Welle und ist auf Level 1 bis 5 begrenzt.
- GameManager spawnt Gegner aus einer Queue im Abstand von 500 ms.

7.3 Turmplatzierung, Upgrade und Verkauf

- Türme werden per Drag-and-Drop auf das Canvas gezogen. PlayLevelInteraction übergibt die TowerID an das ViewModel.
- GameManager.PlaceTower prüft Gold, Pfadfelder und bereits belegte Felder. Danach wird der Turm auf das Raster geschnappt.
- Beim Upgrade werden die Werte des bestehenden Tower-Objekts durch die ausgewählten TowerData-Werte ersetzt.
- Beim Verkauf entfernt GameManager den Turm aus placedTowers und erstattet 70 Prozent der aktuellen Kosten.

7.4 Kampf- und Effektlogik

- Im Rendering-Loop prüft GameManager pro Turm, ob ein lebender Gegner innerhalb der PixelRange liegt.
- Der Angriff wird durch AttackCooldown begrenzt. Dieser berechnet sich aus $0.5 / \text{AttackSpeed}$.
- AoE-Schaden trifft alle Gegner innerhalb von $\text{AoERadius} * \text{tileSize}$ um das Einschlagszentrum.
- Verwundbarkeit und Resistenz werden über EnemyData mit dem Turmtyp verglichen und verändern den Schaden prozentual.
- Slow wird bei erneutem Treffer aufgefrischt, aber nicht mehrfach gestapelt. Burn verursacht Schaden über Zeit, Stun setzt CurrentSpeed auf 0.

7.5 Gegnerbewegung und Lebenspunkte

Enemy bewegt sich entlang der PathTiles des Levels. Die Position wird pro Frame anhand von CurrentSpeed und deltaTime berechnet. Erreicht ein Gegner das Ende des Pfades, verliert der Spieler Lebenspunkte in Höhe von EnemyData.Damage. Stirbt ein Gegner, wird sein GoldDrop dem Spieler gutgeschrieben. Die HealthBar wird nach jedem Positions- oder Lebenspunktupdate aktualisiert.

7.6 Highscore und Game Over

Wenn PlayerHealth auf 0 fällt, aktualisiert HighscoreManager die Tabelle Highscores. Danach publiziert das ViewModel ein GameOverEvent, wodurch MainViewModel zum GameOverScreen navigiert. Der gespeicherte Wert ist die zuletzt vollständig erreichte Welle.

8. Ressourcen und Assets

Ressource	Verwendung
Resources/Enemies/*.png	Gegnergrafiken für EnemyData.TexturePath, inklusive Boss.png.
Resources/Towers/*.png	Turmgrafiken, die über Tower.TexturePath geladen werden.
Resources/MapTiles/*.png	Pfad- und Boden-Tiles für GameManager.InitializeGrid.
Images/*.png	Startbild, Levelvorschauen und Game-Guide-Grafiken.
Levels/*.json	Pfadkoordinaten der einzelnen Level.